Pruebas de escritorio: ejecutar el código a mano

Clase 17 · Jueves 5 de noviembre de 2026 · Aula

Paso	Línea	Se ejecuta	total
1	2	`total = 0`	**0**
2	3	`for precio in lista_precios:`	0
3	4	`total = total + precio`	**120**
4	3	`for precio in lista_precios:`	120
5	4	`total = total + precio`	**370**
6	3	`for precio in lista_precios:`	370
7	5	`if total < 500:`	370

Vistazo al mundo laboral

El reto técnico (coding challenge): la prueba práctica de casi todo proceso de selección en software.

1 · Contacto

Un reclutador te escribe (LinkedIn, bolsa de trabajo)

→

2 · Entrevista RH

Experiencia, expectativas, disponibilidad

→

3 · Reto técnico

Resuelves un problema de programación

→

4 · Con el equipo

Entrevista técnica: explicas tu solución

→

5 · Oferta

Sueldo, prestaciones, fecha de inicio

Tres formatos comunes del reto técnico

En vivo (live coding)

Programas frente al entrevistador, en una plataforma en línea (tipo LeetCode o HackerRank), con tiempo limitado.

Para llevar (take-home)

Te mandan un repositorio con el enunciado. Lo descargas, instalas sus dependencias, lo resuelves en tu computadora en unos días y lo entregas.

En pareja (pair programming)

Programas junto con alguien del equipo, explicando en voz alta lo que piensas.

⚠️ En un take-home, al instalar y correr el repositorio **ejecutas en tu computadora código que escribió otra persona.** ¿Y si la empresa no existe?

Noticia tech

Binance: un reto técnico falso fue parte del camino a un robo de unos 6,300 millones de dólares (29 oct 2026).

JUNIO

Una herramienta falsa

Un ingeniero del equipo de wallets descarga una herramienta alterada desde un anuncio. Binance la detecta, pero los atacantes ya conocen al equipo.

SEPTIEMBRE

El reto técnico falso

A otro ingeniero que buscaba trabajo lo entrevista el «CEO» de una empresa que no existe. El reto: descargar un repositorio. Lo instala en su computadora del trabajo y el antivirus no detecta nada.

20 DE OCTUBRE

Se mueven por dentro

Desde esa computadora toman las de otros ingenieros. Una tenía un token para desplegar sistemas.

29 DE OCTUBRE

El robo

Un script llega al servicio que firma las transacciones: salen unos 6,300 millones de dólares en criptomonedas en pocas horas.

Cómo cuidarte cuando hagas un reto técnico

✓ Nunca lo corras en la computadora de tu trabajo ni donde tengas tus contraseñas.

✓ Úsalo en un entorno aislado: máquina virtual, contenedor o Codespaces.

✓ Lee el código antes de instalar: los scripts de instalación también se ejecutan.

✓ Verifica que la empresa y el reclutador existan, por sus canales oficiales.

Fuentes: reporte de Binance (3 nov 2026); NBC News, Bloomberg y Axios (3 nov 2026). El FBI lo atribuye a TraderTraitor (Corea del Norte), que lo niega.

Objetivo de la sesión

Ejecutar un algoritmo a mano, renglón por renglón, y descubrir si hace lo que dice el enunciado.

1 · La tabla → **2 · Condiciones combinadas** → **3 · Práctica a mano**

Qué es una prueba de escritorio, qué columnas lleva y en qué orden se llena.

and, or, not y paréntesis: el orden en que Python los evalúa y los errores más comunes.

Seis funciones: condiciones combinadas, contadores, acumuladores, if / elif y máximos.

Por qué hoy

En el laboratorio de «Encontrar el bug» les fue bien en general, pero los errores se concentraron en las condiciones combinadas: `or`, `and`, `not` y paréntesis. La prueba de escritorio es la herramienta para encontrar justo esos errores: el código corre, pero la condición no pregunta lo que creías.

¿Qué es una prueba de escritorio?

También se le llama traza o, en inglés, desk check o trace table.

Una prueba de escritorio es **ejecutar un algoritmo a mano**, con **datos concretos**, anotando en una tabla cómo cambia cada variable **línea por línea**, para comprobar si el resultado es el que se esperaba.

Encuentra errores de lógica

El código corre sin marcar error… y aun así da el resultado equivocado. La tabla muestra en qué renglón se desvía.

Te obliga a pensar como Python

Python no adivina lo que quisiste decir: hace exactamente lo que está escrito, en el orden en que está escrito.

No necesita computadora

Sirve en un examen, en una entrevista de trabajo o para entender código que escribió otra persona.

En el laboratorio la plataforma llenaba la tabla por ustedes. Hoy la llenan ustedes, a mano, con las mismas columnas.

Las columnas de la tabla

Las mismas que en la prueba de escritorio del laboratorio. Cada renglón es una línea que Python ejecutó.

Paso	Línea	Se ejecuta	una por variable
Contador de renglones: 1, 2, 3… Empieza en 1 en cada llamada.	Número de la línea que se ejecutó.	El texto de esa línea. Si es un return, se agrega ↩ y el valor que regresa.	También los parámetros. Su valor después de ejecutar la línea; el que cambió se marca.

Reglas

1. Un renglón por cada línea que se ejecuta, en el orden en que Python la ejecuta, no en el orden en que está escrita (un for regresa hacia arriba).

2. En cada renglón escribe el valor de todas las variables como quedan después de esa línea, y marca el que cambió (en el laboratorio sale en negritas azules).

3. Los parámetros también llevan columna: su valor viene de la llamada y ya aparece en el primer renglón.

4. No hay columna para la condición: si el if dio True, el siguiente renglón es una línea de adentro del if; si dio False, se la salta. else no lleva renglón.

5. Al llegar a un return, escribe ↩ y lo que regresa: la función termina y ya no hay más renglones.

Cómo hacerla, paso a paso

El mismo procedimiento para cualquier función.

1

Numera las líneas

Así cada renglón de la tabla dice qué línea se ejecutó.

2

Escribe la llamada y lo esperado

Los datos de entrada y, leyendo el enunciado, qué debería regresar.

3

Dibuja las columnas

Paso, Línea, Se ejecuta y una por variable (parámetros incluidos).

4

Ejecuta renglón por renglón

Una línea a la vez. En un if, resuelve la condición aparte para saber qué línea sigue.

5

Compara

Si lo que regresa no es lo esperado, el primer renglón que se desvía señala el bug.

Elige bien los datos. Con unos datos el código puede funcionar y con otros no (lo vieron en el laboratorio: «con estos datos sí da el resultado correcto»). Prueba un caso normal y uno que pase por cada rama del if.

Ejemplo resuelto: acumulador + if

Tienda en línea: suma los precios del carrito; si el total es menor a 500, se cobran 99 de envío.

Python

```python
def calcular_total_compra(lista_precios):
    total = 0
    for precio in lista_precios:
        total = total + precio
    if total < 500:
        total = total + 99
    return total
```

Llamada

`calcular_total_compra([120, 250])`

Se esperaba

120 + 250 = 370; como es menor a 500, se suman 99 → **469**

Columnas de tu tabla

`Paso · Línea · Se ejecuta · lista_precios · total · precio`

A mano: llena la tabla renglón por renglón; en la siguiente diapositiva la revisamos.

Ejemplo resuelto · solución (1 de 2)

Python

```python
def calcular_total_compra(lista_precios):
    total = 0
    for precio in lista_precios:
        total = total + precio
    if total < 500:
        total = total + 99
    return total
```

Se esperaba

`calcular_total_compra([120, 250])`

120 + 250 = 370 < 500 → 370 + 99 = **469**

Paso	Línea	Se ejecuta	lista_precios	total	precio
Llamada: `calcular_total_compra([120, 250])`					
1	2	`total = 0`	[120, 250]	**0**	
2	3	`for precio in lista_precios:`	[120, 250]	0	**120**
3	4	`total = total + precio`	[120, 250]	**120**	120
4	3	`for precio in lista_precios:`	[120, 250]	120	**250**
5	4	`total = total + precio`	[120, 250]	**370**	250

Ejemplo resuelto · solución (2 de 2)

Python

```python
def calcular_total_compra(lista_precios):
    total = 0
    for precio in lista_precios:
        total = total + precio
    if total < 500:
        total = total + 99
    return total
```

Se esperaba

`calcular_total_compra([120, 250])`

120 + 250 = 370 < 500 → 370 + 99 = **469**

Paso	Línea	Se ejecuta	lista_precios	total	precio
Llamada: `calcular_total_compra([120, 250])` (continúa)					
6	3	`for precio in lista_precios:`	[120, 250]	370	250
7	5	`if total < 500:`	[120, 250]	370	250
8	6	`total = total + 99`	[120, 250]	**469**	250
9	7	`return total` ↩ **469**	[120, 250]	469	250

✓ Regresa 469: coincide con lo esperado

La línea 3 aparece tres veces: dos veces entra al ciclo y la tercera ya no hay más precios, así que salta a la línea 5.

Condiciones combinadas

and, or, not y paréntesis: lo que más costó en «Encontrar el bug»

`and` `or` `not` `( )`

and, or, not: qué regresan

Repaso rápido. Los tres trabajan con valores True / False.

and

a	b	a and b
True	True	**True**
True	False	**False**
False	True	**False**
False	False	**False**

Sólo da True si los dos son True.

or

a	b	a or b
True	True	**True**
True	False	**True**
False	True	**True**
False	False	**False**

Da True si al menos uno es True.

not

a	not a
True	**False**
False	**True**

Invierte el valor.

Cortocircuito

and se detiene en el primer False; **or** se detiene en el primer True. Lo que queda a la derecha ya no se evalúa.

Truco para evaluar rápido

En un **and**, basta un False para que todo sea False. En un **or**, basta un True para que todo sea True.

¿Qué se evalúa primero? La precedencia

Si no hay paréntesis, Python sigue este orden, de arriba hacia abajo.

1	`( )`	Paréntesis	`lo de adentro primero`
2	`+ - * /`	Aritmética	`total + 99`
3	`== != < > <= >= in`	Comparaciones	`edad >= 18`
4	`not`	Negación	`not es_socio`
5	`and`	Y	`a and b`
6	`or`	O	`a or b`

Igual que en aritmética

```
2 + 3 * 4      →  2 + (3 * 4) = 14
a or b and c   →  a or (b and c)
not a and b    →  (not a) and b
not x == 5     →  not (x == 5)
```

and es el × de la lógica; or es el +

Por eso and se resuelve antes que or, aunque esté escrito después.

Regla práctica: si mezclas and con or, pon paréntesis aunque no hagan falta.

Evaluar una condición a mano

La tabla no tiene columna para la condición: la resuelves aparte para saber qué línea sigue.

La condición · con es_vip = False, edad = 16, tiene_adeudo = False

```
if es_vip or edad >= 18 and not tiene_adeudo:
```

paso	qué haces	cómo queda
1	Pon los paréntesis que pone Python	`es_vip or ((edad >= 18) and (not tiene_adeudo))`
2	Sustituye los valores	`False or ((16 >= 18) and (not False))`
3	Resuelve las comparaciones	`False or (False and (not False))`
4	Resuelve los not	`False or (False and True)`
5	Resuelve los and	`False or False`
6	Resuelve los or	**`False`**

✗ La condición da False: no entra al if

Es menor de edad, así que no importa que no tenga adeudo.

Los errores que más vi en el laboratorio

Los cuatro corren sin marcar error. Sólo una prueba de escritorio (o un resultado raro) los delata.

1 · and cuando se quería or

```
✗ es_fin_de_semana and es_festivo
```

Descuento en fin de semana o festivo: and pide las dos a la vez.

```
✓ es_fin_de_semana or es_festivo
```

2 · El not sólo toca lo de junto

```
✗ not es_socio or es_vip
```

Es (not es_socio) or es_vip.

```
✓ not (es_socio or es_vip)
```

3 · Mezclar and con or sin paréntesis

```
✗ es_vip or edad >= 18 and boleto
```

Python lo lee es_vip or (edad >= 18 and boleto).

```
✓ (es_vip or edad >= 18) and boleto
```

4 · Rango con or

```
✗ if x > 0 or x < 10:
```

Siempre es True: todo número cumple una de las dos.

```
✓ if x > 0 and x < 10:   o   if 0 < x < 10:
```

Negar una condición combinada: leyes de De Morgan

Al meter el not a los paréntesis, cada and se vuelve or y cada or se vuelve and.

`not (a and b)`	=	`not a or not b`
no (mayor y con boleto)		menor o sin boleto

`not (a or b)`	=	`not a and not b`
no (lluvia o frío)		sin lluvia y sin frío

✗ **Error típico:** `not (a and b)` escrito como `not a and not b`
Eso pide que no se cumpla ninguna de las dos; la original sólo pide que no se cumplan las dos a la vez.

Práctica a mano

1. Copia la llamada y dibuja la tabla en tu cuaderno.
2. Llénala renglón por renglón, sin computadora.
3. Compara lo que regresa con lo que pide el enunciado: ¿hay bug?

Ejercicio 1 · ¿Dan lo mismo?

Club de esports: para entrar hay que ser socio y, además, ser mayor de edad o tener permiso de un adulto.

Versión A

```
1  def decidir_acceso_a(edad, tiene_permiso, es_socio):
2      if edad >= 18 or tiene_permiso and es_socio:
3          return "entra"
4      return "no entra"
```

Versión B

```
1  def decidir_acceso_b(edad, tiene_permiso, es_socio):
2      if (edad >= 18 or tiene_permiso) and es_socio:
3          return "entra"
4      return "no entra"
```

Llamadas

```
(20, False, False)
(15, True, True)
```

Se esperaba

(20, False, False) → **"no entra"**
no es socio
(15, True, True) → **"entra"**

Columnas de tu tabla

`Paso · Línea · Se ejecuta · edad · tiene_permiso · es_socio`

A mano: llena la tabla renglón por renglón; en la siguiente diapositiva la revisamos.

Ejercicio 1 · solución, versión A

Python

```python
def decidir_acceso_a(edad, tiene_permiso, es_socio):
    if edad >= 18 or tiene_permiso and es_socio:
        return "entra"
    return "no entra"
```

Se esperaba

(20, False, False) → **"no entra"**

(15, True, True) → **"entra"**

Paso	Línea	Se ejecuta	edad	tiene_permiso	es_socio
Llamada: `decidir_acceso_a(20, False, False)`					
1	2	`if edad >= 18 or tiene_permiso and es_socio:`	20	False	False
2	3	`return "entra"` ↩ `'entra'`	20	False	False
Llamada: `decidir_acceso_a(15, True, True)`					
1	2	`if edad >= 18 or tiene_permiso and es_socio:`	15	True	True
2	3	`return "entra"` ↩ `'entra'`	15	True	True

✗ Con (20, False, False) la versión A regresa "entra": deja pasar a alguien que no es socio

Python la lee como `edad >= 18 or (tiene_permiso and es_socio)`: con 20 años, el or ya es True.

Ejercicio 1 · solución, versión B

Python

```python
def decidir_acceso_b(edad, tiene_permiso, es_socio):
    if (edad >= 18 or tiene_permiso) and es_socio:
        return "entra"
    return "no entra"
```

Se esperaba

(20, False, False) → **"no entra"**

(15, True, True) → **"entra"**

Paso	Línea	Se ejecuta	edad	tiene_permiso	es_socio
Llamada: `decidir_acceso_b(20, False, False)`					
1	2	`if (edad >= 18 or tiene_permiso) and es_socio:`	20	False	False
2	4	`return "no entra"` ↩ `'no entra'`	20	False	False
Llamada: `decidir_acceso_b(15, True, True)`					
1	2	`if (edad >= 18 or tiene_permiso) and es_socio:`	15	True	True
2	3	`return "entra"` ↩ `'entra'`	15	True	True

✓ La versión B es la correcta: regresa "no entra" y "entra"

Con (15, True, True) las dos versiones coinciden: un solo caso de prueba no basta para encontrar el bug.

Ejercicio 2 · Contador con not

Radio: cuenta las canciones que se pueden programar. No deben ser muy cortas (menos de 120 s) ni muy largas (más de 300 s).

Python

```python
def contar_canciones_para_radio(lista_duraciones):
    contador = 0
    for duracion in lista_duraciones:
        if not duracion < 120 or duracion > 300:
            contador = contador + 1
    return contador
```

Llamada

```
contar_canciones_para_radio([200, 90, 400])
```

Se esperaba

1 (sólo la de 200 s)

Columnas de tu tabla

```
Paso · Línea · Se ejecuta · lista_duraciones · contador · duracion
```

A mano: llena la tabla renglón por renglón; en la siguiente diapositiva la revisamos.

Ejercicio 2 · solución (1 de 2)

Python

```python
def contar_canciones_para_radio(lista_duraciones):
    contador = 0
    for duracion in lista_duraciones:
        if not duracion < 120 or duracion > 300:
            contador = contador + 1
    return contador
```

Se esperaba

1 (sólo la de 200 s)

Paso	Línea	Se ejecuta	lista_duraciones	contador	duracion
Llamada: `contar_canciones_para_radio([200, 90, 400])`					
1	2	`contador = 0`	[200, 90, 400]	**0**	
2	3	`for duracion in lista_duraciones:`	[200, 90, 400]	0	**200**
3	4	`if not duracion < 120 or duracion > 300:`	[200, 90, 400]	0	200
4	5	`contador = contador + 1`	[200, 90, 400]	**1**	200
5	3	`for duracion in lista_duraciones:`	[200, 90, 400]	1	**90**
6	4	`if not duracion < 120 or duracion > 300:`	[200, 90, 400]	1	90

Ejercicio 2 · solución (2 de 2)

Python

```python
def contar_canciones_para_radio(lista_duraciones):
    contador = 0
    for duracion in lista_duraciones:
        if not duracion < 120 or duracion > 300:
            contador = contador + 1
    return contador
```

Se esperaba

1 (sólo la de 200 s)

Paso	Línea	Se ejecuta	lista_duraciones	contador	duracion
Llamada: `contar_canciones_para_radio([200, 90, 400])` (continúa)					
7	3	`for duracion in lista_duraciones:`	[200, 90, 400]	1	**400**
8	4	`if not duracion < 120 or duracion > 300:`	[200, 90, 400]	1	400
9	5	`contador = contador + 1`	[200, 90, 400]	**2**	400
10	3	`for duracion in lista_duraciones:`	[200, 90, 400]	2	400
11	6	`return contador` ↩ **2**	[200, 90, 400]	2	400

✗ Regresa 2, pero se esperaba 1

Con 400 también se ejecuta la línea 5: el not sólo niega `duracion < 120`. Corrección: `not (duracion < 120 or duracion > 300)`

Ejercicio 3 · not, and, or

Ciberseguridad: alerta si hay 3 o más intentos fallidos y, además, la IP es desconocida o es fuera del horario laboral.

Python

```python
def es_alerta_seguridad(intentos, es_ip_conocida, es_horario_laboral):
    if intentos >= 3 and not es_ip_conocida or not es_horario_laboral:
        return True
    return False
```

Llamadas

```
es_alerta_seguridad(1, True, False)
es_alerta_seguridad(5, False, True)
```

Se esperaba

(1, True, False) → **False** (no llega a 3)

(5, False, True) → **True**

Columnas de tu tabla

```
Paso · Línea · Se ejecuta · intentos · es_ip_conocida · es_horario_laboral
```

A mano: llena la tabla renglón por renglón; en la siguiente diapositiva la revisamos.

Ejercicio 3 · solución

Python

```python
def es_alerta_seguridad(intentos, es_ip_conocida, es_horario_laboral):
    if intentos >= 3 and not es_ip_conocida or not es_horario_laboral:
        return True
    return False
```

Se esperaba

(1, True, False) → **False**

(5, False, True) → **True**

Paso	Línea	Se ejecuta	intentos	es_ip_conocida	es_horario_laboral
Llamada: `es_alerta_seguridad(1, True, False)`					
1	2	`if intentos >= 3 and not es_ip_conocida or not es_horario_laboral:`	1	True	False
2	3	`return True` ↩ **True**	1	True	False
Llamada: `es_alerta_seguridad(5, False, True)`					
1	2	`if intentos >= 3 and not es_ip_conocida or not es_horario_laboral:`	5	False	True
2	3	`return True` ↩ **True**	5	False	True

✗ La primera llamada lanza una alerta con sólo 1 intento fallido

Faltan paréntesis. Corrección: `intentos >= 3 and (not es_ip_conocida or not es_horario_laboral)`

Ejercicio 4 · if / elif / else

Paquetería: el costo depende del peso, del tipo de envío y de si el cliente es premium.

Python

```python
def calcular_costo_envio(peso, es_express, es_premium):
    if peso > 20 and not es_premium:
        costo = 300
    elif es_express or peso > 10:
        costo = 150
    else:
        costo = 80
    if es_premium and es_express:
        costo = costo - 50
    return costo
```

Llamadas

```
calcular_costo_envio(25, True, True)
calcular_costo_envio(8, False, False)
```

Se esperaba

¿Cuánto cuesta cada envío? Fíjate qué líneas no aparecen en tu tabla.

Columnas de tu tabla

`Paso · Línea · Se ejecuta · peso · es_express · es_premium · costo`

A mano: llena la tabla renglón por renglón; en la siguiente diapositiva la revisamos.

Ejercicio 4 · solución (1 de 2)

Python

```python
def calcular_costo_envio(peso, es_express, es_premium):
    if peso > 20 and not es_premium:
        costo = 300
    elif es_express or peso > 10:
        costo = 150
    else:
        costo = 80
    if es_premium and es_express:
        costo = costo - 50
    return costo
```

Se esperaba

¿Cuánto cuesta cada envío?

Paso	Línea	Se ejecuta	peso	es_express	es_premium	costo
Llamada: `calcular_costo_envio(25, True, True)`						
1	2	`if peso > 20 and not es_premium:`	25	True	True	
2	4	`elif es_express or peso > 10:`	25	True	True	
3	5	`costo = 150`	25	True	True	**150**
4	8	`if es_premium and es_express:`	25	True	True	150
5	9	`costo = costo - 50`	25	True	True	**100**
6	10	`return costo` ↩ **100**	25	True	True	100

Ejercicio 4 · solución (2 de 2)

Python

```python
1  def calcular_costo_envio(peso, es_express, es_premium):
2      if peso > 20 and not es_premium:
3          costo = 300
4      elif es_express or peso > 10:
5          costo = 150
6      else:
7          costo = 80
8      if es_premium and es_express:
9          costo = costo - 50
10     return costo
```

✓ Regresa 100 y 80

Las líneas 3 y 6 nunca aparecen: el else no lleva renglón, y la 3 sólo se ejecuta si la condición de la línea 2 da True.

Paso	Línea	Se ejecuta	peso	es_express	es_premium	costo
Llamada: `calcular_costo_envio(8, False, False)`						
1	2	`if peso > 20 and not es_premium:`	8	False	False	
2	4	`elif es_express or peso > 10:`	8	False	False	
3	7	`costo = 80`	8	False	False	**80**
4	8	`if es_premium and es_express:`	8	False	False	80
5	10	`return costo` ↩ **80**	8	False	False	80

Ejercicio 5 · Acumulador + contador

Escolar: el promedio sólo de las calificaciones aprobatorias (de 6 a 10).

Python

```python
def calcular_promedio_aprobadas(lista_calificaciones):
    suma = 0
    cantidad = 0
    for calificacion in lista_calificaciones:
        if calificacion >= 6 and calificacion <= 10:
            suma = suma + calificacion
            cantidad = cantidad + 1
    return suma / cantidad
```

Llamada

`calcular_promedio_aprobadas([9, 5, 7])`

Se esperaba

(9 + 7) / 2 = **8.0** (el 5 no cuenta)

Columnas de tu tabla

`Paso · Línea · Se ejecuta · lista_calificaciones · suma · cantidad · calificacion`

A mano: llena la tabla renglón por renglón; en la siguiente diapositiva la revisamos.

Ejercicio 5 · solución (1 de 2)

Python

```python
def calcular_promedio_aprobadas(lista_calificaciones):
    suma = 0
    cantidad = 0
    for calificacion in lista_calificaciones:
        if calificacion >= 6 and calificacion <= 10:
            suma = suma + calificacion
            cantidad = cantidad + 1
    return suma / cantidad
```

Se esperaba

(9 + 7) / 2 = **8.0** (el 5 no cuenta)

Paso	Línea	Se ejecuta	lista_calificaciones	suma	cantidad	calificacion
Llamada: calcular_promedio_aprobadas([9, 5, 7])						
1	2	suma = 0	[9, 5, 7]	**0**		
2	3	cantidad = 0	[9, 5, 7]	0	**0**	
3	4	for calificacion in lista_calificaciones:	[9, 5, 7]	0	0	**9**
4	5	if calificacion >= 6 and calificacion <= 10:	[9, 5, 7]	0	0	9
5	6	suma = suma + calificacion	[9, 5, 7]	**9**	0	9
6	7	cantidad = cantidad + 1	[9, 5, 7]	9	**1**	9
7	4	for calificacion in lista_calificaciones:	[9, 5, 7]	9	1	**5**

Ejercicio 5 · solución (2 de 2)

Python

```python
def calcular_promedio_aprobadas(lista_calificaciones):
    suma = 0
    cantidad = 0
    for calificacion in lista_calificaciones:
        if calificacion >= 6 and calificacion <= 10:
            suma = suma + calificacion
            cantidad = cantidad + 1
    return suma / cantidad
```

✓ Regresa 8.0: coincide

Paso 8: con el 5, después de la línea 5 sigue la 4 (no entró al if). Si ninguna aprueba, la línea 8 divide entre 0.

Paso	Línea	Se ejecuta	lista_calificaciones	suma	cantidad	calificacion
Llamada:	`calcular_promedio_aprobadas([9, 5, 7])` (continúa)					
8	5	`if calificacion >= 6 and calificacion <= 10:`	[9, 5, 7]	9	1	5
9	4	`for calificacion in lista_calificaciones:`	[9, 5, 7]	9	1	**7**
10	5	`if calificacion >= 6 and calificacion <= 10:`	[9, 5, 7]	9	1	7
11	6	`suma = suma + calificacion`	[9, 5, 7]	**16**	1	7
12	7	`cantidad = cantidad + 1`	[9, 5, 7]	16	**2**	7
13	4	`for calificacion in lista_calificaciones:`	[9, 5, 7]	16	2	7
14	8	`return suma / cantidad` ↩ **8.0**	[9, 5, 7]	16	2	7

Ejercicio 6 · Máximo con not

Sensor de temperatura: la lectura más alta, ignorando las inválidas (menores a 0 o mayores a 60).

Python

```python
def encontrar_lectura_maxima(lista_lecturas):
    maxima = 0
    for lectura in lista_lecturas:
        if not (lectura < 0 or lectura > 60) and lectura > maxima:
            maxima = lectura
    return maxima
```

Llamada

```
encontrar_lectura_maxima([25, 99, 31])
```

Se esperaba

31 (el 99 es una lectura inválida)

Columnas de tu tabla

```
Paso · Línea · Se ejecuta ·
lista_lecturas · maxima · lectura
```

A mano: llena la tabla renglón por renglón; en la siguiente diapositiva la revisamos.

Ejercicio 6 · solución (1 de 2)

Python

```python
def encontrar_lectura_maxima(lista_lecturas):
    maxima = 0
    for lectura in lista_lecturas:
        if not (lectura < 0 or lectura > 60) and lectura > maxima:
            maxima = lectura
    return maxima
```

Se esperaba

31 (el 99 es inválida)

Paso	Línea	Se ejecuta	lista_lecturas	maxima	lectura
Llamada: encontrar_lectura_maxima([25, 99, 31])					
1	2	maxima = 0	[25, 99, 31]	**0**	
2	3	for lectura in lista_lecturas:	[25, 99, 31]	0	**25**
3	4	if not (lectura < 0 or lectura > 60) and lectura > maxima:	[25, 99, 31]	0	25
4	5	maxima = lectura	[25, 99, 31]	**25**	25
5	3	for lectura in lista_lecturas:	[25, 99, 31]	25	**99**
6	4	if not (lectura < 0 or lectura > 60) and lectura > maxima:	[25, 99, 31]	25	99

Ejercicio 6 · solución (2 de 2)

Python

```python
def encontrar_lectura_maxima(lista_lecturas):
    maxima = 0
    for lectura in lista_lecturas:
        if not (lectura < 0 or lectura > 60) and lectura > maxima:
            maxima = lectura
    return maxima
```

Se esperaba

31 (el 99 es inválida)

Paso	Línea	Se ejecuta	lista_lecturas	maxima	lectura
Llamada: `encontrar_lectura_maxima([25, 99, 31])` (continúa)					
7	3	`for lectura in lista_lecturas:`	[25, 99, 31]	25	**31**
8	4	`if not (lectura < 0 or lectura > 60) and lectura > maxima:`	[25, 99, 31]	25	31
9	5	`maxima = lectura`	[25, 99, 31]	**31**	31
10	3	`for lectura in lista_lecturas:`	[25, 99, 31]	31	31
11	6	`return maxima` ↩ **31**	[25, 99, 31]	31	31

✓ Regresa 31: coincide

Paso 6: con el 99 sigue la línea 3 (no entró): `not (False or True)` → False. Equivale a `lectura >= 0 and lectura <= 60`.

Para llevar

La tabla

Un renglón por línea ejecutada, con el valor de todas las variables; marca el que cambió.

Las condiciones

Resuélvelas aparte: sustituye los valores, pon los paréntesis y decide qué línea sigue.

La precedencia

not, luego and, luego or. Si mezclas and con or, usa paréntesis.

Los datos

Prueba más de un caso: uno que funcione no garantiza que todos funcionen.

Python hace lo que escribiste, no lo que quisiste decir. La prueba de escritorio muestra la diferencia.